

BOUNCEBOX

Simulateur de Billard 2D

Analyse · Conception · Implémentation en Python

Tobias Geoffray & Ugo Royer

Promo 2028 · 14 Juin 2026

Présentation du Projet

01

Qu'est-ce que BounceBox ?

Jeu de billard à deux joueurs développé en Python.

- Simulation physique réaliste : déplacements, rebonds, frottements et collisions élastiques
- Partie en tours alternés : Joueur Rouge vs Joueur Bleu
- Minuteur par tour (45 s) → contrainte stratégique
- Victoire : premier à atteindre 5 points
- Mode solo contre le Bot IA disponible

Défis techniques



Physique 2D temps réel



Intelligence Artificielle



Interface PyQt5 réactive



Rafraichissement fluide (60 FPS)



Persistance des données CSV

Architecture Globale

02

Modèle du jeu

Boule, Boule_blanche
Boule_de_couleur

Plateau

Joueur

Partie

Impact

Interface graphique

GameWidget
(rendu graphique)

GameModeDialog
(paramètres)

SettingsDialog
(difficulté bot)

Trajectoire

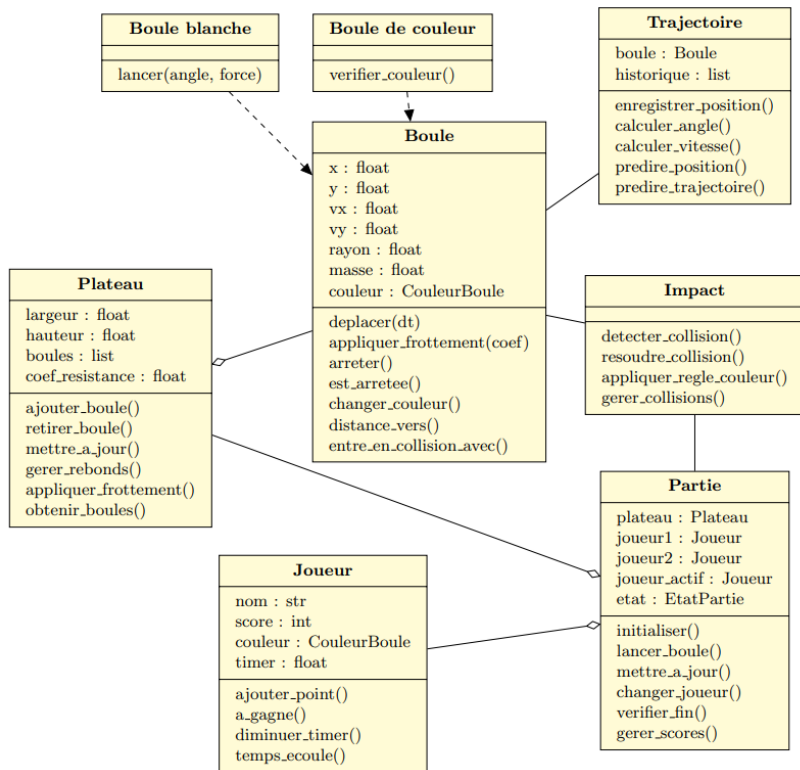
Contrôle de l'affichage

MainWindow
(orchestration UI)

GameThread
(boucle jeu / 60 FPS)

Diagramme de classe UML

03



Classe Impact

Détection :

Distance euclidienne entre deux centres :

$$d = \sqrt{(\Delta x^2 + \Delta y^2)} < r_1 + r_2$$

Résolution (collision élastique) :

Conservation de la quantité de mouvement :

```
Impact.resoudre_collision_elastique(b1, b2)
```

Règles couleur après collision :

Grise → prend la couleur du joueur

Du joueur → point marqué, retirée

Adverse → redevient grise

Classe Plateau

largeur, hauteur

Dimensions de l'espace de jeu

boules : list

Liste de toutes les boules

coef_resistance

Frottement appliqué chaque frame

ajouter_boule()

Ajout d'une boule au plateau

mettre_a_jour()

Boule principale → déplace, rebondit, freine

gerer_rebonds()

Inversion de vitesse aux bords

Zoom : La Classe Boule

05

Boule (classe parente)

```
x, y, vx, vy, rayon, masse  
couleur : CouleurBoule  
deplacer(dt) · arreter() ·  
distance_vers()
```

Boule_blanche

Toujours de couleur BLANCHE
lancer(angle, force)
→ Calcule vx/vy par projection trigonométrique

Boule_de_couleur

Rejette la couleur BLANCHE
(lève ValueError si tentative)
verifier_couleur() — validation typologique

CouleurBoule :

BLANCHE

GRISE

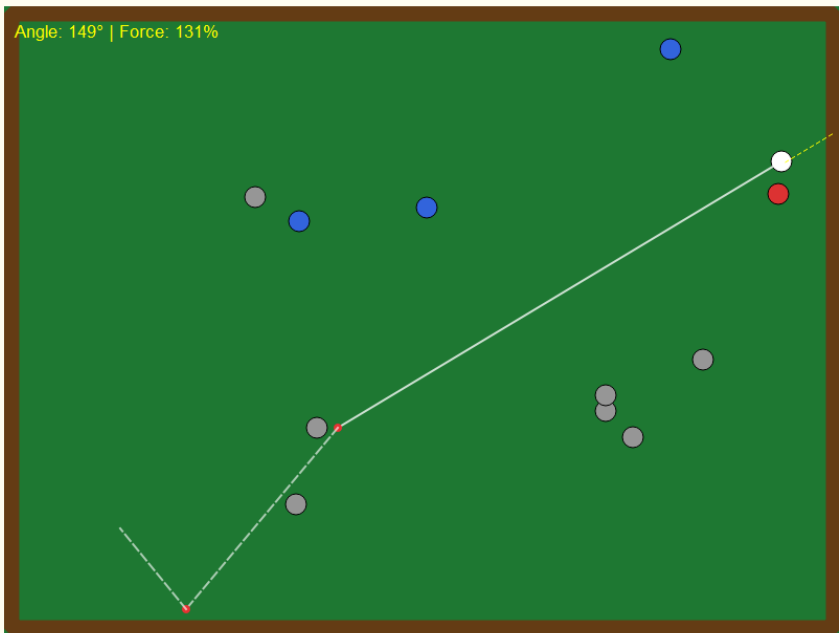
ROUGE

BLEUE

Récurtivité & Prédiction de Trajectoire

06

Simulation visuelle



`calculer_trajectoire_recursive()`

- 1. Calculer** — Le segment courant à partir de la position et de la vitesse
- 2. Détecter** — Si collision avec un mur ou une boule
- 3. Rebondir** — Inverser composante vx ou vy + frottement avec la conservation de la quantité de mouvement
- 4. Rappel récursif** — Même fonction avec nouvelle pos/vitesse
- 5. Condition d'arrêt** — Nombre de rebonds max atteint ici fixes à 2

72

angles
(0°→360°, pas 5°)

50

puissances
(10%→500%, pas 10%)

3 600

coups évalués
max par tour

≥ 6

score → sortie
anticipée

Algorithme Brute-Force Optimisé

1. Filtrage des angles valides
(`_construire_angles_valides`)
2. Pour chaque (angle, puissance) valide :
 - Copie du plateau + simulation physique complète
 - Attribution d'un score aux collisions
3. Early exit si score ≥ 6
4. Retourner le meilleur coup trouvé

Système de Scoring

Boule grise **+1 pt**

Boule adverse **+2 pts**

Boule du bot **+4 pts**

Méthode des images (_construire_angles_valides)

Objectif : éviter de simuler des angles inutiles (ne pointant vers aucune boule)

1

Images virtuelles

Pour chaque boule cible, calculer 4 réflexions symétriques par rapport aux 4 murs du plateau (méthode des images optiques).

2

Intervalle angulaire

Pour chaque boule réelle et image, calculer l'arc angulaire depuis la boule blanche qui permettrait de l'atteindre : $\arcsin(\text{rayon} / \text{distance})$.

3

Accumulation

Ces intervalles sont accumulés dans un tableau de différences — construction efficace de la liste des angles valides (sans boucle naïve sur tous les indices).

4

Résultat

Seuls les angles « prometteurs » sont explorés en profondeur, réduisant drastiquement le nombre de simulations physiques complètes.

Intégration dans la boucle de jeu

```
Joueur ← est_bot : bool  
        bot : Bot
```

GameThread à chaque frame :

```
if joueur_actif.est_bot and not bot_played:  
    coup = Bot.calculer_meilleur_coup(plateau)  
    Partie.lancer_boule_blanche(*coup)  
    bot_played = True
```

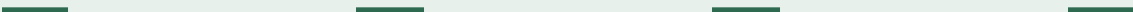
Paramétrage de la Difficulté

pas_angle et **pas_puissance** sont réglables
dans SettingsDialog avant chaque partie.

Niveau	Pas angle	Pas puiss.	Temps
Facile	10°	20%	~2-3s
Normal	5°	10%	~10-15s
Difficile	2°	5%	>45s ⚠

Flux de jeu bot

Tour du bot



Interface Graphique & Avantages du QThread

10

MainWindow

`main_window.py`

Fenêtre principale
Orchestre UI + connecte signaux Qt
Gère Nouvelle Partie / Quitter

GameThread

`game_thread.py`

Boucle de jeu 60 FPS
Isolée dans QThread
Émet 5 signaux Qt

GameWidget

`game_widget.py`

Rendu plateau PyQt5
Saisie souris (clic-glissé)
Prédiction trajectoire 2 rebonds

GameModeDialog SettingsDialog

`game_mode_dialog.py`

Choix mode de jeu
Difficulté du bot
Avant chaque partie

 **QThread** → Calcul physique isolé → IHM jamais figée → **60 FPS constants**

Stockage des Données

11

En mémoire (Partie)

```
historique_tours : list[dict]
```

- Numéro du tour
- Joueur actif
- Temps de jeu (s)
- Points gagnés
- Score cumulé des joueurs

fin
de
partie

Avantages & Perspectives

- ✓ CSV simple, lisible, compatible Excel pour analyse externe
- ✓ Mode ajout → historique multi-parties dans un seul fichier

🔮 *Perspective : base SQL pour structuration et analyses avancées*

Export CSV (partie.py)

Mode ajout → conservation de plusieurs parties dans un même fichier

=====						
PARTIE DU 14/06/2026 À 20:25						
=====						
--- RESUME DE LA PARTIE ---						
Joueur Rouge	Joueur 1	Score Final	4			
Joueur Bleu	Joueur 2	Score Final	5			
VAINQUEUR		Joueur 2				
TEMPS TOTAL			236.0			
--- CHRONOLOGIE TOUR PAR TOUR ---						
Tour	Joueur Actif	Couleur	Temps (s)	Bille(s) Gagné	Total Rouge	Total Bleu
Tour 1	Joueur 1	rouge	28.1	0	0	0
Tour 2	Joueur 2	bleue	21.4	1	0	1
Tour 3	Joueur 1	rouge	19.9	1	1	1
Tour 4	Joueur 2	bleue	19.3	1	1	2
Tour 5	Joueur 1	rouge	26.7	2	3	2
Tour 6	Joueur 2	bleue	19.1	1	3	3
Tour 7	Joueur 1	rouge	23.3	0	3	3
Tour 8	Joueur 2	bleue	18.7	0	3	3
Tour 9	Joueur 1	rouge	18.8	0	3	3
Tour 10	Joueur 2	bleue	18.3	1	3	4
Tour 11	Joueur 1	rouge	22.2	1	4	4

Bilan & Conclusion

✓ Points forts

- Modèle physique stable (60 FPS)
- IHM ergonomique (PyQt5)
- Code modulaire MVC
- IA compétitive (bot)
- Figures imposées toutes respectées
- Tests unitaires complets

🌟 Perspectives

- IA Minimax (jeu optimal)
- Améliorer la prediction des trajectoires
- Collisions multiples simultanées
- Base de données SQL
- Animations & effets visuels

🎮 Démonstration en direct

Merci de votre attention !
Avez-vous des questions ?